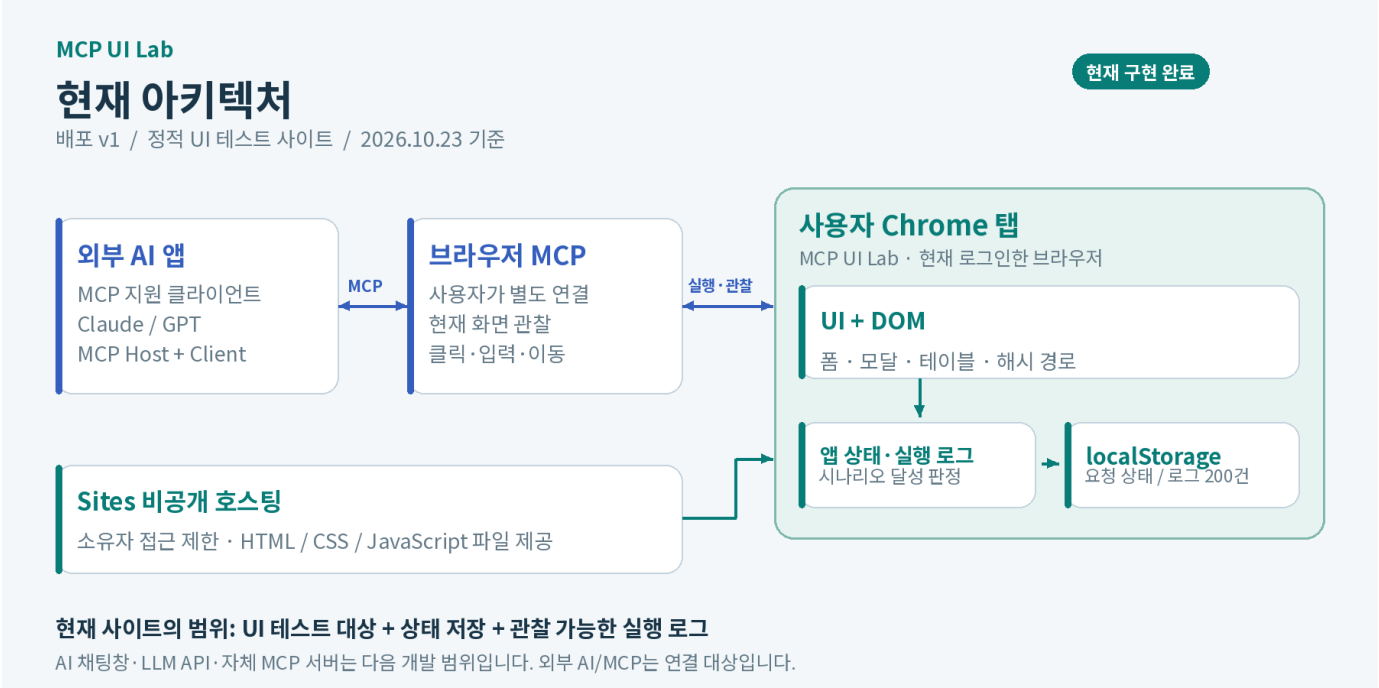
현재 프로젝트와 아키텍처

2026.10.23 · 배포 v1 · 비공개 소유자 전용 · 현재 구현과 확장 제안을 구분

현재 배포본은 외부 Claude/GPT와 브라우저 MCP가 조작할 수 있는 UI 테스트 사이트입니다. 사이트 내부 채팅을 통한 UI 조작은 다음 개발 범위입니다.



현재 기술 스펙

구분	현재 배포 v1
프론트엔드	HTML5 + CSS + Vanilla JavaScript (strict mode, IIFE)
프레임워크·빌드	Vue/React 미사용 · package.json 및 빌드 의존성 없음
라우팅	location.hash + hashchange · 브라우저 뒤로 가기 지원
저장소	localStorage, 키: mcp-ui-lab-v1 · 서버 DB 없음
호스팅·접근	Sites 정적 dist/ 제공 · 소유자 1인 허용 · 외부 방문자 0명
로그·응답	브라우저 이벤트 로그 최대 200건 · JSON 다운로드 · 1.5초 모의 응답
AI/MCP	브라우저 자동화 대상 UI 제공 · 내부 채팅/LLM API/자체 MCP 서버 미구현

사이트: https://mcp-ui-lab.wintryhorse5.chatgpt.site

소스 기준: 90f78fd1367408acfb8a7ba9c9b275e01bc0a358 · 배포 v1의 commit과 로컬 HEAD 일치. 소유자 1인 허용, 외부 방문자 0명. 공개 전환은 소유자 확인 필요.

기능과 자동화 테스트

UI 계약 · 화면 경로 · 6개 시나리오

현재 기능

영역	현재 동작
입력 폼	이름 필수·최대 60자, 메모 최대 300자, 유형·우선순위·저장 동의
제출·검증	이름 공백 또는 동의 미체크 시 차단, 오류·성공 표시, 목록 추가
모달	native <dialog>, 열기·취소·확인·Escape, 닫힌 후 포커스 복원
테이블	이름·ID 검색, 상태 필터, 이름 정렬, 행·현재 페이지 선택, 완료 처리, 4건씩 페이지
페이지 이동	실습·시나리오·목록·상세·가이드, hash 경로와 브라우저 기록
지연 응답	1500ms 타이머, loading 중 비활성, success 또는 error 표시
관찰·보조	로그 필터·삭제·JSON 내보내기, 달성 상태, 명령 복사, 초기화 확인

6개 시나리오

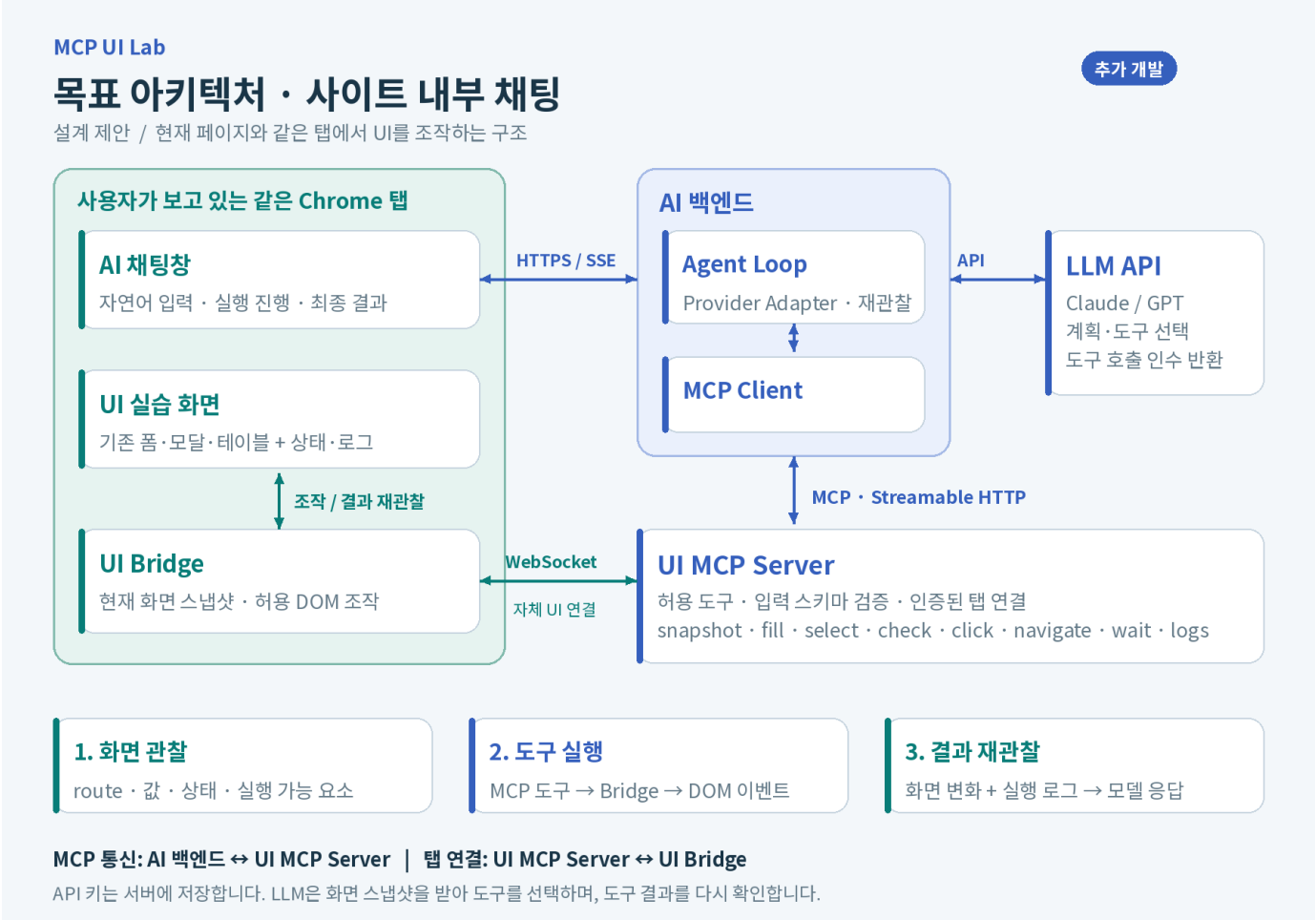
ID	명령 핵심	확인할 결과
form	정해진 이름·유형·우선순위·메모 입력 후 동의·제출	MCP 자동화 테스트 / integration / high / 브라우저 MCP 실습, 신규 요청
validation	이름 비우기 + 동의 해제 후 제출	두 오류 메시지 표시, 요청 개수 유지
modal	열기 → 취소 → 재열기 → 확인	대화상자 닫힘, modal.cancel / modal.confirm
table	로그인 검색 → 이름 정렬 → 선택 완료 → 다음 페이지	REQ-1042가 완료, 다음 페이지 표시
navigation	REQ-1042 상세 열기 → 목록 복귀	#record/REQ-1042 제목·ID 일치 후 #pages
async	성공 요청 후 실패 요청	loading → success, loading → error / 버튼 비활성 해제

주요 경로: #lab · #scenarios · #pages · #record/REQ-1042 · #guide. 동적 요소는 row-REQ-1042, select-REQ-1042, detail-REQ-1042 형태로 식별합니다.

자동화는 role/label과 id/data-testid 계약을 활용합니다. 전체 식별자와 선택값은 함께 제공한 Markdown 개발 명세에 정리했습니다.

내부 AI 채팅 확장 설계

원래 목표 A · 사용자가 보고 있는 현재 Chrome 탭을 연결하는 제안



실행 흐름

채팅 명령 → 현재 화면 관찰 → LLM 도구 선택 → Agent Loop의 MCP 호출 → UI Bridge의 DOM 조작 → 결과 재관찰 → 채팅 응답. LLM API 키와 MCP Client는 서버에서 관리합니다.

구성	개발 제안	역할
프론트엔드	1차는 현재 정적 UI 유지, 확장 시 Vue 3 + TypeScript + Vite	사용자의 Vue 경험을 반영한 전환 후보
AI 백엔드	Node.js + TypeScript	Provider Adapter, Agent Loop, 인증·진행 이벤트
MCP	공식 TypeScript SDK, 호환 버전 고정	도구 목록/호출, 입력 스키마와 결과 검증
연결	HTTPS + SSE / MCP Streamable HTTP / WebSocket	채팅 진행 / MCP / 현재 브라우저 탭 연결을 구분
상태·검증	초기 localStorage 유지, Playwright E2E 추가	기존 6개 시나리오를 회귀 기준으로 유지

처음에는 정적 UI를 유지해 연결을 검증합니다. Vue 전환은 별도 단계로 진행하고, SDK·런타임 버전은 구현 시 호환성을 확인해 lockfile에 고정합니다.

MCP 도구와 화면 정보 계약

이 페이지의 도구·JSON·검증 방식은 미구현 설계 제안

도구 제안	주요 입력	결과·제약
ui.get_snapshot	관찰 범위(선택)	route, revision, 요소·값·상태. 채팅/비밀값 제외
ui.fill	targetId, value, expectedRevision	허용 input/textarea만, input/change 이벤트
ui.select	targetId, value, expectedRevision	등록된 select와 허용 option만
ui.check	targetId, checked, expectedRevision	checkbox만, 이미 같은 값이면 유지
ui.click	targetId, expectedRevision	표시·활성 상태 확인 후 허용 버튼 클릭
ui.navigate	route, expectedRevision	허용된 내부 hash 경로만
ui.wait_for	targetId, condition, timeoutMs	상태/표시/비활성 조건을 관찰, 무한 대기 금지
ui.get_logs	since, limit	관련 실행 로그만 반환, 기존 최대 200건 정책

화면 스냅샷

route, snapshotId, revision, 표시 요소의 targetId·role·label·value·options·enabled·visible, 최근 로그를 전달합니다. 전체 HTML을 매번 보내지 않고 앱 전용 JSON으로 현재 상태를 관찰합니다. 채팅창·비밀값은 제외합니다.

```
{
 "snapshotId": "snap-0007",
 "revision": 7,
 "route": "#lab",
 "elements": [
 {
 "targetId": "request-name",
 "role": "textbox",
 "value": "",
 "enabled": true,
 "visible": true
 }
 ]
}
```

명령과 결과 연결

명령: commandId + requestId + tool + arguments. 쓰기 인수에는 expectedRevision을 포함합니다. 결과: commandId + status + observed + revision + eventIds. 이벤트 전달 후 실제 DOM 변화와 로그를 관찰해야 성공으로 처리합니다.

사용자·세션·탭은 인증된 연결에서 서버가 결정합니다. 오래된 화면이면 재관찰하고, 중복 commandId는 재실행하지 않습니다. 연결 종료와 응답 유실 시 제출 성공 여부를 먼저 확인해 이중 등록을 막습니다.

Bridge는 허용한 현재 페이지 DOM만 조작합니다. 합성 이벤트는 실제 사용자 입력과 동일하다고 간주하지 않습니다. 다른 탭·사이트·브라우저 자체 조작과 신뢰된 제스처가 필요한 기능은 이 Bridge의 범위 밖입니다.

데이터·로그와 검증 근거

현재 코드에서 확인한 상태 저장과 테스트 범위

브라우저 상태 모델

항목	내용
rows	id, name, category, priority, status, note. 초기 8건, 다음 ID REQ-1043.
form	name, category, priority, note, consent. 이름 60자, 메모 300자.
logs	id, timestamp(ISO), type, event, message, detail. 최신순 최대 200건.
completed / nextId	달성 시나리오 ID 목록 / 다음 요청 번호.
저장 범위	localStorage의 mcp-ui-lab-v1. 기기·프로필 간 동기화 없음.
메모리 전용	검색·필터·정렬·페이지·선택과 시나리오 중간 증거 플래그.

로그 계약

유형은 info / success / error입니다. form.submit, form.validation, modal.confirm, table.bulk_complete, navigation.change, async.success/error, scenario.complete 등으로 UI 결과를 관찰합니다.

JSON 내보내기: application, exportedAt, completedScenarios, events. 로그 삭제는 달성 상태를 유지합니다. 세션 초기화 확인 시 기본 8건과 미달성 상태로 복구하고 session.reset을 기록합니다.

검증 기록과 한계

기존 QA.md(2026-10-22)에는 지원 브라우저 조작(CUA)으로 6개 시나리오를 수행해 6/6 달성을 확인한 기록이 있습니다. 390px 반응형, 상태 보존, JSON 파싱, 명령 복사, 필터와 초기화도 점검했습니다.

이번 문서는 배포 메타데이터와 소스를 검토해 작성했습니다. 브라우저 시나리오를 다시 실행하거나 외부 Claude/GPT MCP 연결을 새로 검증하지 않았습니다. 외부 모델·MCP의 end-to-end 검증은 남아 있습니다.

앱의 달성 배지는 테스트 근거의 일부입니다. 예를 들어 페이지 왕복은 상세 진입 후 목록 복귀를 감지하므로 특정 ID·제목의 일치까지 별도 E2E 단언으로 확인해야 합니다.

응답 실습의 200/500 표시는 1500ms 타이머 기반 모의 결과이며 실제 HTTP 상태 코드가 아닙니다.

개발 순서와 완료 기준

Claude Code 개발 기준 · 현재 UI를 유지하면서 한 단계씩 연결

단계	작업	완료 기준
0. 기준 유지	현재 v1 소스와 6개 시나리오를 기준점으로 고정	기존 UI·식별자·로그 동작 확인
1. 현재 탭 Bridge	요소 등록부·스냅샷·허용 DOM 조작·탭 연결	LLM 없이 연결 도구로 폼 1개 등록, 화면·로그 일치
2. MCP 도구	MCP 서버·클라이언트·스키마·실행 결과	MCP 도구 탐색·호출로 같은 탭의 6개 흐름 실행
3. 내부 AI 채팅	Agent Loop·Provider Adapter·중지·진행 표시	내부 명령 → 도구 실행 → 실제 화면 확인 → 결과 답변
4. 통합 검증·배포	외부 모델 통합·Playwright 회귀·세션 격리	두 탭 격리, 연결 끊김, 실패·재시도 확인 후 비공개 배포

최소 완료 기준

1. 내부 채팅 명령으로 같은 탭에 실제 입력값과 신규 요청 행이 나타난다.

2. 최종 답변이 실제 DOM 상태와 실행 로그에 근거한다.

3. 6개 시나리오를 MCP 통합 테스트와 별도 Playwright 회귀로 확인한다.

4. 두 탭 격리, 새로고침, 연결 종료, 중지, 시간 초과와 중복 실행을 처리한다.

5. 소유자 전용 접근과 서버 API 키 보관을 유지한다.

배포 결정

현재 Sites는 정적 배포입니다. Node 백엔드 제안은 장기 연결을 지원하는 별도 호스트가 필요합니다. Sites 서버 통합을 선택하면 Worker에 맞춘 서버와 세션 구성을 별도로 설계합니다. 정적 파일에 서버 코드를 추가하는 것만으로 실행되지는 않습니다.

Claude Code 첫 작업

기존 dist/와 README.md, QA.md를 읽고 UI·식별자·로그 계약을 유지하세요. LLM 없이 현재 탭 Bridge를 먼저 구현해 snapshot과 폼 조작을 검증한 뒤 MCP를 연결하세요. 그다음 내부 채팅과 Provider Adapter를 추가하세요. 상세 명세와 복사 가능한 시작 지시문은 Markdown에 있습니다.

근거·참고

프로젝트: dist/index.html, dist/app.js, dist/style.css, README.md, QA.md, .openai/hosting.json. 공식 문서는 MCP 역할·전송 방식과 Playwright 식별자 사용의 근거이며, Bridge와 개발 단계는 프로젝트 설계 제안입니다.

https://ts.sdk.modelcontextprotocol.io/

https://ts.sdk.modelcontextprotocol.io/v2/clients/connect.html

https://ts.sdk.modelcontextprotocol.io/v2/serving/http.html

https://playwright.dev/docs/locators